# EDA Automation & Reproducible Reports

Workshop 4 — Module 9

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

# Module Roadmap

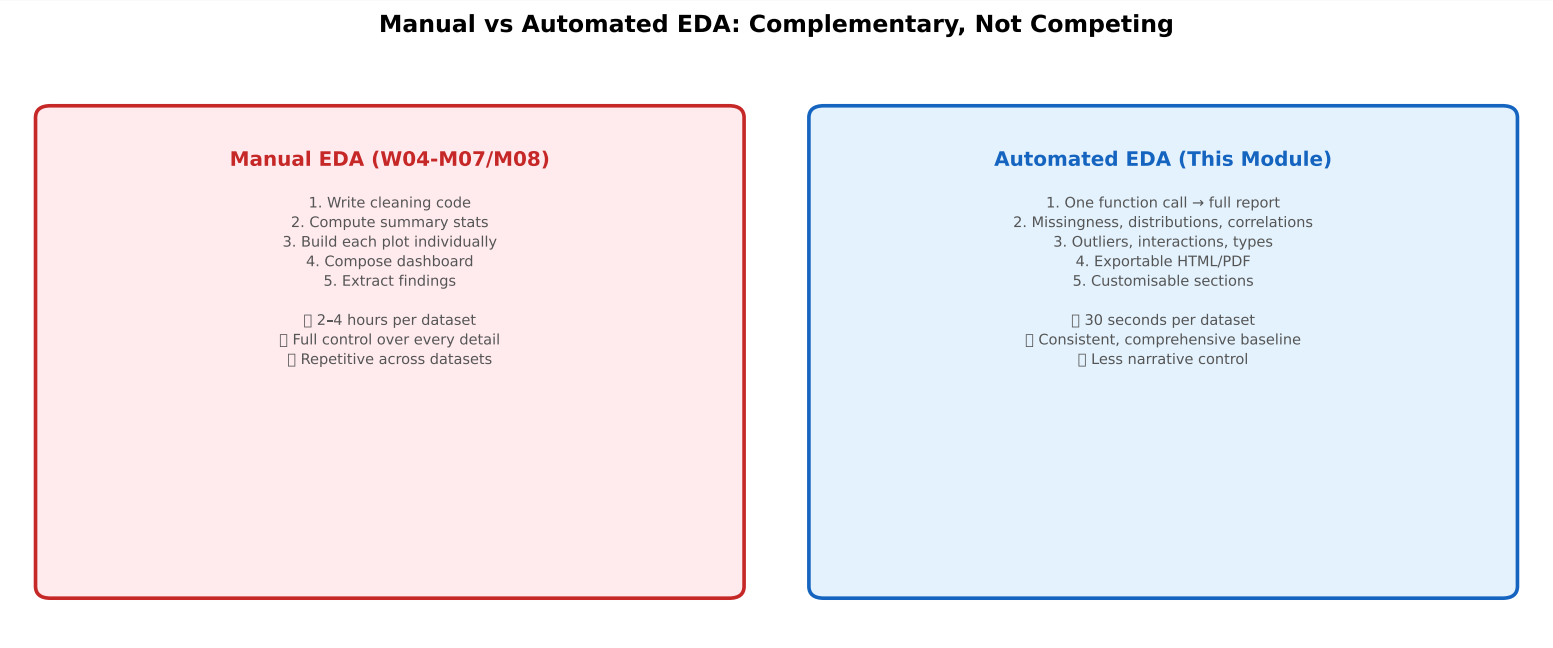
1. Manual vs automated EDA: complementary roles
2. Auto-EDA packages: skimr, DataExplorer, ydata-profiling
3. The hybrid workflow: auto baseline $\rightarrow$ manual deep-dive
4. Reproducible reports: Quarto, RMarkdown, Jupyter
5. Parameterised reports: one template, many outputs

**Learning Outcome**

You will generate a comprehensive automated EDA report in under 60 seconds, identify which findings require manual follow-up, and produce a reproducible report that combines code, prose, and output in a single document.

# Manual vs Automated

# Two Approaches, One Goal



# When to Use Which

| Situation | Approach | Why |
|---|---|---|
| First look at unknown data | Automated | Fast, comprehensive baseline |
| Client presentation | Manual | Full narrative control |
| 20 datasets to profile | Automated | Batch processing in a loop |
| Deep-dive on one finding | Manual | Custom chart for the story |
| Coursework / publication | Both | Auto for audit, manual for report |

**Pro-Tip**

Auto-EDA is a **screening tool**, not a replacement for thinking. It tells you *what to look at*; you decide *what it means*. The hybrid workflow uses auto-EDA for the first 30 seconds, then spends 2 hours on the findings that matter.

# Auto-EDA Packages

# The Package Landscape

**Automated EDA Package Landscape: R & Python**

| Package | Lang | Call | Features | Best For |
|---|---|---|---|---|
| `skimr` | R | `skim(df)`<br>`→ console summary` | Compact stats per type; integrates with dplyr pipes | **Best for quick console inspection** |
| `DataExplorer` | R | `create_report(df)`<br>`→ HTML report` | Distributions, correlations, missing, PCA, all automatic | **Best auto-report for R** |
| `ydata-profiling` | Python | `ProfileReport(df)`<br>`→ HTML report` | Most comprehensive; alerts, interactions, missing | **Gold standard for Python EDA** |
| `sweetviz` | Python | `sv.analyze(df)`<br>`→ HTML report` | Side-by-side comparison (train vs test, groups) | **Best for comparing two datasets** |
| `dtale` | Python | `dtale.show(df)`<br>`→ interactive app` | Browser-based interactive EDA with click-to-code | **Best for interactive exploration** |
| `summarytools` | R | `dfSummary(df)`<br>`→ HTML/console` | Frequency tables, graphs embedded in summary | **Best for categorical variables** |

# R: skimr + DataExplorer

```r
#         skimr: compact console summary

library(skimr)
skim(apps)                      # full summary
skim(apps) |> summary()         # just counts

# Grouped skim (like group_by + summarise but for EDA)
apps |> group_by(Type) |> skim(Rating, Reviews)

#         DataExplorer: one-click HTML report

library(DataExplorer)
create_report(apps)             # full HTML report
# Or individual plots:
plot_missing(apps)              # missingness bar
plot_histogram(apps)            # all numeric histograms
plot_correlation(apps)          # correlation heatmap
plot_bar(apps)                  # all categorical bars

#         summarytools: freq tables + embedded graphs
library(summarytools)
dfSummary(apps) |> view()       # opens in browser
```

# Python: ydata-profiling + sweetviz

```python
#         ydata-profiling (formerly pandas-profiling)
# pip install ydata-profiling
from ydata_profiling import ProfileReport

report = ProfileReport(
    apps,
    title="Google Play Store EDA",
    explorative=True,            # interactions tab
    correlations={
        "pearson": True,
        "spearman": True,
        "kendall": False})

report.to_file("eda_report.html")        # standalone HTML
report.to_notebook_iframe()               # inline in Jupyter

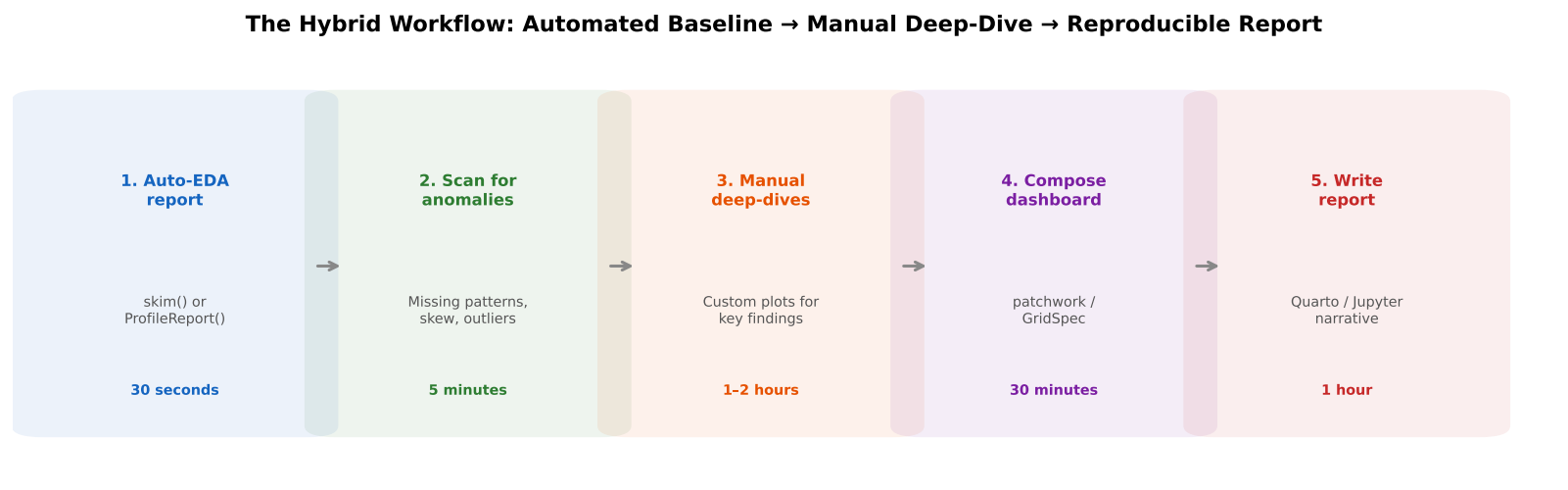
#         sweetviz: compare two datasets

# pip install sweetviz
import sweetviz as sv

# Compare Free vs Paid
report = sv.compare(
    [apps.query("Type=='Free'"), "Free"],
    [apps.query("Type=='Paid'"), "Paid"])
report.show_html("free_vs_paid.html")
```

# The Hybrid Workflow

# Auto Baseline → Manual Deep-Dive



# What to Look for in the Auto Report

| Section | Scan For | Follow-Up Action |
|---|---|---|
| Overview | Row/col counts, types | Verify expected dime |
| Alerts | High correlation, skew, zeros, constants | Investigate each alert |
| Missing | % per column, patterns | Shadow analysis if $>$ |
| Distributions | Skew, bimodality, spikes | Manual histogram wi |
| Correlations | $\lvert r \rvert > 0.7$ | Scatter plot for top p |
| Interactions | Unexpected nonlinearities | Faceted scatter + sm |

The auto report generates $\sim$50 plots in 30 seconds. Your job: scan for the 3–5

that require a custom, publication-quality deep-dive.

# Reproducible Reports

# Code + Prose + Output = One Document

**Reproducible Report Formats: Code + Prose + Output in One Document**

| Format | Language | Origin | Features | Best For |
|---|---|---|---|---|
| `Quarto (.qmd)` | R + Python | 2022+ (Posit) | Unified: renders to HTML, PDF, revealjs, docx, dashboard. Supports R, Python, Julia. | **Next-gen standard; replaces RMarkdown** |
| `RMarkdown (.Rmd)` | R (+ Python via reticulate) | 2014+ (Posit) | knitr + pandoc. Mature, huge ecosystem. Paramaterised reports. | **Legacy R standard; still widely used** |
| `Jupyter (.ipynb)` | Python (+ R via IRkernel) | 2014+ (Project Jupyter) | Cell-based notebook. Inline plots, markdown. Widgets for interactivity. | **Python standard; teaching & prototyping** |
| `Observable (.ojs)` | JavaScript | 2021+ (Observable Inc) | Reactive cells. D3-powered. Web-native. Quarto can embed .ojs cells. | **Web dashboards; interactive articles** |

# Quarto: The Modern Standard

```
---
title: "EDA Report: Google Play Store"
author: "Endri Raco"
format:
  html:
    toc: true
    code-fold: true
    theme: cosmo
  pdf:
    documentclass: article
execute:
  echo: true
  warning: false
---

## Data Cleaning
```{r}
library(tidyverse)
apps <- read_csv("googleplaystore.csv") |>
  distinct(App, .keep_all = TRUE) |>
  mutate(Reviews = as.numeric(Reviews))
```

## Distribution
```{r}
#| fig-cap: "Rating distribution (left-skewed)"
```
```

# Jupyter Notebook: Python Standard

```python
# Cell 1: Markdown
# # EDA Report: Google Play Store
# **Author**: Endri Raco | **Date**: 2025

# Cell 2: Code
import pandas as pd
apps = pd.read_csv("googleplaystore.csv")
apps.describe()

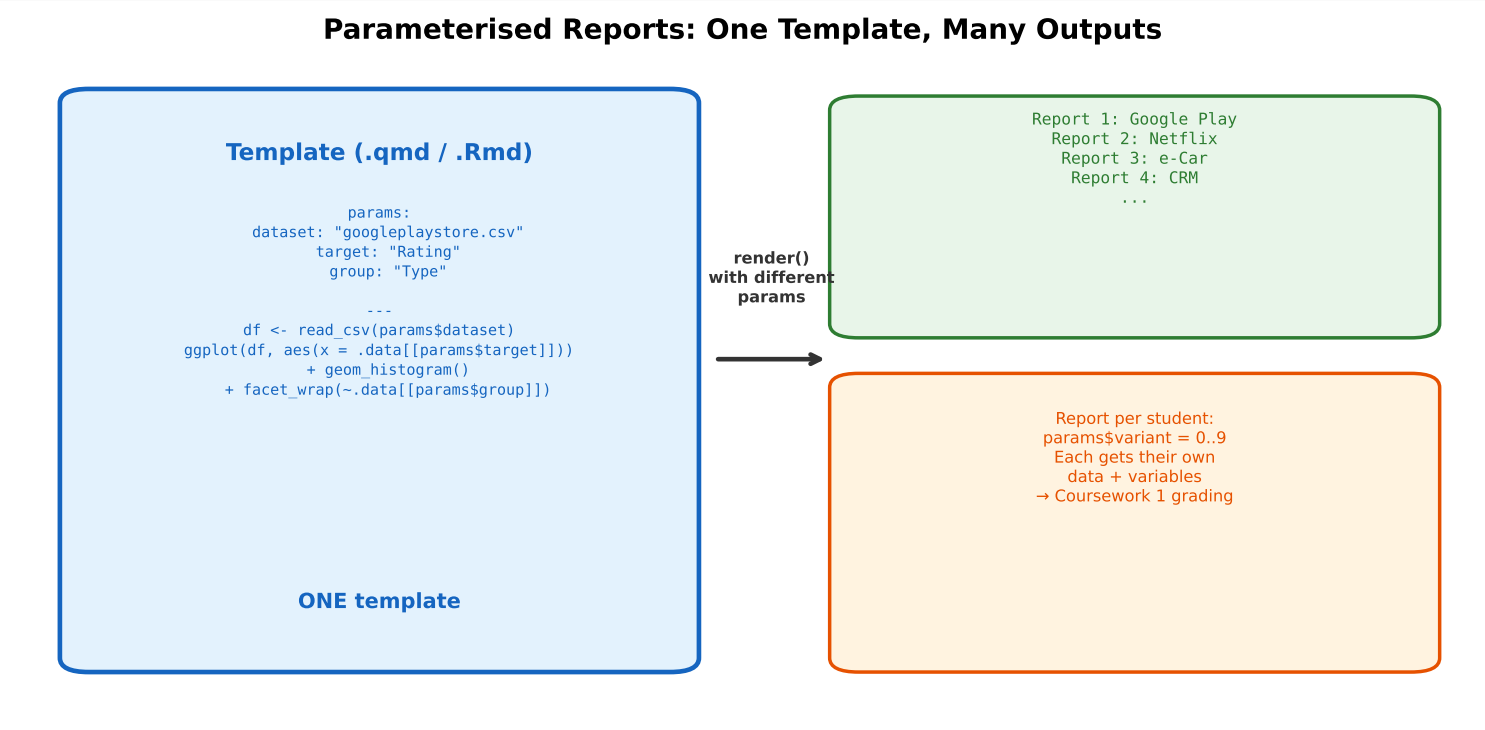
# Cell 3: Visualization
apps["Rating"].hist(bins=30, color="#1565C0")
plt.title("Rating Distribution (left-skewed)")
plt.show()

# Cell 4: Markdown
# ## Key Finding
# Rating is left-skewed (mean = 4.17, median = 4.30).

# Export:
# jupyter nbconvert --to pdf notebook.ipynb
# jupyter nbconvert --to html notebook.ipynb
# quarto render notebook.ipynb --to pdf
```

# Parameterised Reports

# One Template, Many Outputs



# Parameterised Quarto / RMarkdown

```
---
title: "EDA Report"
params:
  dataset: "googleplaystore.csv"
  target: "Rating"
  group: "Type"
---
```{r}
df <- read_csv(params$dataset) |> drop_na(!!sym(params$target))
ggplot(df, aes(x = .data[[params$target]])) +
  geom_histogram(bins = 30) +
  facet_wrap(reformulate(params$group))
```
```

```
# Render for each dataset:
quarto::quarto_render("template.qmd",
  execute_params = list(dataset = "netflix.csv",
    target = "duration", group = "type"))

# Batch render all coursework variants:
for (v in 0:9) {
  quarto::quarto_render("template.qmd",
    output_file = paste0("report_v", v, ".html"),
    execute_params = list(variant = v))
}
```

# Why Reproducibility Matters

| Benefit | How |
|---|---|
| Auditability | Reviewer re-runs your code, gets identical output |
| Version control | `git diff` shows exactly what changed |
| Error correction | Fix one line, re-render; no manual copy-paste |
| Scalability | Same report for 10 datasets via parameterisation |
| Collaboration | Co-authors edit code, not screenshots |
| Publication | Journals increasingly require reproducible code |

**Pro-Tip**

The coursework requires R *and* Python scripts plus a report. A Quarto document with both {`r`} and {`python`} code chunks can produce the entire submission as **one reproducible file**. The `reticulate` package

# Synthesis

# Key Takeaways

1. **Auto-EDA is a screening tool**: 30-second baseline that tells you what to investigate manually.
2. In R: `skimr::skim()` for console, `DataExplorer::create_report()` for HTML. In Python: `ydata-profiling` for comprehensive reports, `sweetviz` for comparisons.
3. The **hybrid workflow**: auto report $\rightarrow$ scan for anomalies $\rightarrow$ manual deep-dive $\rightarrow$ compose dashboard $\rightarrow$ write narrative.
4. **Quarto** (.qmd) is the modern standard for reproducible reports: code + prose + output in one file, renders to HTML, PDF, slides, dashboards.
5. **Parameterised reports** generate many outputs from one template — ideal for batch EDA and coursework grading.
6. **Reproducibility** = auditability + version control + error correction + scalability.

# Essential Readings

- Quarto documentation: `https://quarto.org`
- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapters 28–29 (Quarto).
- ydata-profiling: `https://docs.profiling.ydata.ai`
- Peng, R. (2011). "Reproducible Research in Computational Science." *Science*, 334(6060).

**Next Module**: Lab — Full EDA on Netflix (W04-M10)